

ROTEIRO DE IMPLEMENTAÇÃO EM HARDWARE COM SAÍDA VGA

MODULAÇÕES DIGITAIS UNIPOLARES NRZ-L E NRZ-I NA PLACA BASYS 3

ANA MARTA DE SOUSA SANTOS – 082230041

ALEX AKIO NISHIMURA JUNIOR – 081230007

MARIA EDUARDA FERREIRA BIANCHINI – 081230001

PEDRO HENRIQUE RODRIGUES DE ASSIS – 081230018

PROJETO DE COMUNICAÇÃO DE DADOS E SISTEMAS RECONFIGURÁVEIS

SÃO BERNARDO DO CAMPO

2026

LISTA DE TABELAS

Tabela 1 – Organização dos arquivos por projeto.....	4
Tabela 2 – Parâmetros de temporização do padrão VGA 640×480 @ 60 Hz.....	10
Tabela 3 – Mapeamento dos pinos VGA no arquivo de restrições XDC.....	12
Tabela 4 – Correspondência entre o nível modulado e a cor exibida na saída VGA.....	14
Tabela 5 – Identificação dos sinais nos projetos NRZ-L e NRZ-I com VGA.....	15
Tabela 6 – Comparação entre as modulações NRZ-L e NRZ-I na implementação com VGA...	16

SUMÁRIO

1	INTRODUÇÃO.....	4
2	ORGANIZAÇÃO DOS ARQUIVOS UTILIZADOS	4
3	MÓDULOS VHDL PARA SIMULAÇÃO	5
3.1	Módulo nrz_l.vhd	5
3.2	Módulo nrz_i.vhd	6
3.3	Testbenches	7
4	MÓDULOS DE TOPO COM SAÍDA VGA.....	8
4.1	Módulo top_nrz_l_basys3.vhd	8
4.2	Módulo top_nrz_i_basys3.vhd	9
5	PADRÃO VGA 640×480 E GERAÇÃO DE SINCRONISMO	10
6	ARQUIVO DE RESTRIÇÕES XDC	11
7	CRIAÇÃO DO PROJETO NO VIVADO.....	13
8	ADIÇÃO DOS ARQUIVOS VHDL E RESTRIÇÕES	13
9	CONFIGURAÇÃO DA SIMULAÇÃO	13
9	CONFIGURAÇÃO DA SIMULAÇÃO.....	13
10	EXECUÇÃO DA SIMULAÇÃO.....	14
11	RESULTADOS EM HARDWARE — SAÍDA VGA	14
12	IDENTIFICAÇÃO DOS SINAIS	15
13	EXPLICAÇÃO DO COMPORTAMENTO OBSERVADO	15
14	COMPARAÇÃO ENTRE NRZ-L E NRZ-I.....	16
15	INTERPRETAÇÃO FINAL DOS RESULTADOS.....	17
	REFERÊNCIAS BIBLIOGRÁFICAS	18

1. INTRODUÇÃO

Este roteiro descreve a implementação em hardware das modulações digitais Unipolar NRZ-L e Unipolar NRZ-I na placa de desenvolvimento Basys 3, com a incorporação de uma saída visual via interface VGA. Enquanto as versões anteriores dos projetos utilizavam LEDs como indicadores do nível modulado, esta implementação avançada transmite o estado do sinal diretamente a um monitor externo: quando o sinal modulado está em nível alto, a tela exibe a cor **azul**; quando está em nível baixo, exibe a cor **vermelho**. Essa representação visual torna o comportamento das modulações imediatamente perceptível e intuitivo, pois a alternância de cores na tela corresponde diretamente à alternância dos níveis da onda modulada ao longo do tempo.

O projeto é composto por dois conjuntos de arquivos independentes: um para a simulação comportamental no Vivado, utilizando os módulos *nrz_l.vhd* e *nrz_i.vhd* juntamente com seus respectivos *testbenches*; e outro para a implementação em hardware, utilizando os módulos de topo *top_nrz_l_basys3.vhd* e *top_nrz_i_basys3.vhd*, que integram a lógica de modulação com o gerador de sincronismo VGA e a lógica de colorização. Os módulos de topo foram desenvolvidos de forma autossuficiente - sem instanciar os módulos de simulação - para facilitar a síntese e reduzir a hierarquia do projeto destinado ao hardware.

2. ORGANIZAÇÃO DOS ARQUIVOS UTILIZADOS

Cada projeto é composto por quatro arquivos: o módulo comportamental de simulação, o *testbench* correspondente, o módulo de topo com VGA para implementação em hardware e o arquivo de restrições XDC. Os módulos de simulação e os de hardware são separados - os primeiros descrevem apenas a lógica de modulação com parâmetros de temporização genéricos, enquanto os segundos integram a modulação, o divisor de *clock*, o gerador de sincronismo VGA e a lógica de cores em um único arquivo. A Tabela 1, a seguir, organiza esses arquivos por projeto e função.

Tabela 1 – Organização dos arquivos por projeto

ARQUIVO	PROJETO	TIPO	FUNÇÃO
nrz_l.vhd	NRZ-L	Source VHDL	Módulo comportamental NRZ-L para simulação (genérico, sem VGA).
tb_nrz_l.vhd	NRZ-L	Testbench	Estimula o módulo nrz_l.vhd com a sequência 1011001010101111.

top_nrz_l_basys3.vhd	NRZ-L	Top-level VHDL	Módulo de topo: modulação NRZ-L + gerador VGA + lógica de cores.
basys3_nrz_l.xdc	NRZ-L	Restrições XDC	Mapeia clock, switches, LEDs e pinos VGA da placa Basys 3.
nrz_i.vhd	NRZ-I	Source VHDL	Módulo comportamental NRZ-I para simulação (genérico, sem VGA).
tb_nrz_i.vhd	NRZ-I	Testbench	Estimula o módulo nrz_i.vhd com a sequência 1011001110001111.
top_nrz_i_basys3.vhd	NRZ-I	Top-level VHDL	Módulo de topo: modulação NRZ-I + gerador VGA + lógica de cores.
basys3_nrz_i.xdc	NRZ-I	Restrições XDC	Mapeia clock, switches, LEDs e pinos VGA da placa Basys 3.

Fonte: elaborado pelos autores (2026).

3. MÓDULOS VHDL PARA SIMULAÇÃO

3.1 Módulo nrz_l.vhd

O módulo *nrz_l.vhd* implementa a modulação NRZ-L em nível comportamental, com dois parâmetros genéricos que tornam o circuito facilmente adaptável: *N_BITS*, que define a largura do vetor de entrada (padrão 16), e *CYCLES_PER_BIT*, que determina por quantos ciclos de *clock* cada bit permanece na saída (padrão 5). A transmissão ocorre do bit mais significativo para o menos significativo: ao receber o pulso de *start*, o circuito inicia em *data_in(15)* e avança um bit a cada *CYCLES_PER_BIT* ciclos. A lógica é direta: a saída *out_reg* recebe o valor lógico do bit atual - '1' produz nível alto e '0' produz nível baixo -, sem qualquer dependência do estado anterior. O Código 1, a seguir, apresenta a declaração da entidade e os sinais internos.

Código 1 – Declaração da entidade e sinais internos do módulo nrz_l.vhd

```
entity nrz_l is
  generic (
    N_BITS          : integer := 16;
    CYCLES_PER_BIT : integer := 5
  );
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;
    start    : in  std_logic;
    data_in  : in  std_logic_vector(N_BITS - 1 downto 0);
    nrz_out  : out std_logic;
    bit_indx : out integer range 0 to N_BITS - 1
  );
end nrz_l;
-- Sinais internos de controle
```

```

signal bit_index_reg : integer range 0 to N_BITS - 1 := N_BITS - 1;
signal cycle_count   : integer range 0 to CYCLES_PER_BIT - 1 := 0;
signal active        : std_logic := '0';
signal out_reg       : std_logic := '0';

```

Fonte: elaborado pelos autores (2026).

O sinal *active* controla se a transmissão está em andamento. O sinal *cycle_count* conta os ciclos de *clock* dentro do intervalo de cada bit, e *bit_index_reg* mantém o índice do bit sendo transmitido. O Código 2, a seguir, apresenta o núcleo do processo de transmissão.

Código 2 – Processo de transmissão NRZ-L: atribuição direta do bit à saída (nrz_l.vhd)

```

if start = '1' and active = '0' then
    active      <= '1';
    bit_index_reg <= N_BITS - 1;
    cycle_count  <= 0;
    out_reg      <= data_in(N_BITS - 1);  -- inicia com o MSB

elsif active = '1' then
    if cycle_count = CYCLES_PER_BIT - 1 then
        cycle_count <= 0;
        if bit_index_reg = 0 then
            active <= '0';  -- fim da transmissão
        else
            bit_index_reg <= bit_index_reg - 1;
            out_reg <= data_in(bit_index_reg - 1);  -- NRZ-L: nível = bit
        end if;
    else
        cycle_count <= cycle_count + 1;
    end if;
end if;

```

Fonte: elaborado pelos autores (2026).

3.2 Módulo nrz_i.vhd

O módulo *nrz_i.vhd* implementa a modulação NRZ-I com a mesma estrutura de entidade que o *nrz_l.vhd*: mesmos parâmetros genéricos, mesmas portas e mesmos sinais internos de controle. A única diferença reside na regra de atualização do registrador de saída *out_reg*: o primeiro bit inicializa o nível da saída com o valor de *data_in(15)*; para cada bit subsequente, aplica-se a regra de inversão - se o bit for '1', *out_reg* <= *not out_reg*; se for '0', *out_reg* <= *out_reg*. Essa distinção transforma a informação da modulação: no NRZ-I, o que importa é a **transição** do sinal, não o nível absoluto. O Código 3, a seguir, apresenta essa lógica central.

Código 3 – Lógica de inversão de nível no módulo nrz_i.vhd

```

if start = '1' and active = '0' then

```

```

    active          <= '1';
    bit_index_reg   <= N_BITS - 1;
    cycle_count     <= 0;
    out_reg         <= data_in(N_BITS - 1);    -- primeiro nível = MSB

elsif active = '1' then
    if cycle_count = CYCLES_PER_BIT - 1 then
        cycle_count <= 0;
        if bit_index_reg > 0 then
            bit_index_reg <= bit_index_reg - 1;
            if data_in(bit_index_reg - 1) = '1' then
                out_reg <= not out_reg; -- bit '1': inverte
            else
                out_reg <= out_reg;      -- bit '0': mantém
            end if;
        else
            active <= '0';
        end if;
    else
        cycle_count <= cycle_count + 1;
    end if;
end if;
end if;

```

Fonte: elaborado pelos autores (2026).

3.3 Testbenches

Os *testbenches* *tb_nrz_l.vhd* e *tb_nrz_i.vhd* seguem a mesma estrutura: instanciam o módulo correspondente via *entity work*, geram o *clock* com período de 20 ns (equivalente a 50 MHz) e aplicam a sequência de bits através do processo de estímulo. No *tb_nrz_l.vhd*, a sequência utilizada é *1011001010101111*; no *tb_nrz_i.vhd*, é *1011001110001111*. Com *CLK_PERIOD = 20 ns* e *BIT_PERIOD = 100 ns*, resulta *CYCLES_PER_BIT = 5*, de modo que cada bit permanece estável por 5 ciclos de *clock* na saída. O processo de estímulo aplica *reset* por 40 ns, aguarda 40 ns adicionais, sincroniza com a borda de subida do *clock*, pulsa *start* por um ciclo e aguarda a transmissão completa dos 16 bits (1.600 ns) antes de encerrar a simulação com um *assert failure*. O Código 4, a seguir, apresenta as constantes e a instanciação do módulo no *testbench* NRZ-L.

Código 4 – Constantes e instanciação do módulo no testbench *tb_nrz_l.vhd*

```

constant CLK_PERIOD      : time      := 20 ns;
constant BIT_PERIOD      : time      := 100 ns;
constant CYCLES_PER_BIT : integer := BIT_PERIOD / CLK_PERIOD;    -- = 5
constant INPUT_BITS      : std_logic_vector(15 downto 0) :=
    "1011001010101111";

 uut: entity work.nrz_l
    generic map (
        N_BITS          => N_BITS,
        CYCLES_PER_BIT => CYCLES_PER_BIT
    )

```

```

)
port map (
    clk      => clk,    rst    => rst,    start => start,
    data_in  => data_in,
    nrz_out  => nrz_out,
    bit_indx => bit_indx
);

```

Fonte: elaborado pelos autores (2026).

4. MÓDULOS DE TOPO COM SAÍDA VGA

4.1 Módulo `top_nrz_l_basys3.vhd`

O módulo de topo `top_nrz_l_basys3.vhd` integra em uma única entidade a lógica de modulação NRZ-L, o gerador de sincronismo VGA e a lógica de cores. A entidade expõe seis grupos de portas: o *clock* de 100 MHz da placa (*clk*), os 16 *switches* (*sw[15:0]*), os 16 LEDs (*led[15:0]*), os três vetores de cor (*vgaRed[3:0]*, *vgaGreen[3:0]*, *vgaBlue[3:0]*) e os sinais de sincronismo (*Hsync*, *Vsync*). O Código 5, a seguir, apresenta a declaração da entidade e as constantes de temporização.

Código 5 – Declaração da entidade e constantes de temporização (`top_nrz_l_basys3.vhd`)

```

entity top_nrz_l_basys3 is
    port (
        clk      : in  std_logic;
        sw       : in  std_logic_vector(15 downto 0);
        led      : out std_logic_vector(15 downto 0);
        vgaRed   : out std_logic_vector(3 downto 0);
        vgaGreen : out std_logic_vector(3 downto 0);
        vgaBlue  : out std_logic_vector(3 downto 0);
        Hsync    : out std_logic;
        Vsync    : out std_logic
    );
end top_nrz_l_basys3;

-- Cada bit dura 2 segundos:
-- 100 MHz x 2 s = 200.000.000 ciclos
constant CYCLES_PER_BIT : integer := 200000000;

```

Fonte: elaborado pelos autores (2026).

A lógica de modulação NRZ-L é implementada diretamente no módulo de topo, sem instanciar o módulo `nrz_l.vhd`. O registrador `nrz_out_reg` armazena o nível modulado atual, que é atualizado no início de cada intervalo de bit (quando `cycle_count = 0`) com o valor do *switch* correspondente ao índice atual: `nrz_out_reg <= led_state(bit_index)`. Os LEDs exibem permanentemente os valores dos *switches* (`led <= sw`), servindo como referência visual da sequência de entrada. O Código 6, a seguir, mostra o processo de modulação e avanço de índice.

**Código 6 – Processo de modulação NRZ-L e controle de índice de bit
(top_nrz_l_basys3.vhd)**

```

led_state <= sw;
led       <= led_state;

process(clk)
begin
    if rising_edge(clk) then

        -- Atualiza o nível modulado no início de cada intervalo de bit
        if cycle_count = 0 then
            nrz_out_reg <= led_state(bit_index);  -- NRZ-L: nível = bit
        end if;

        -- Avança o índice ao final de cada intervalo
        if cycle_count = CYCLES_PER_BIT - 1 then
            cycle_count <= 0;
            if bit_index = 0 then
                bit_index <= N_BITS - 1;          -- reinicia em SW15
            else
                bit_index <= bit_index - 1;
            end if;
        else
            cycle_count <= cycle_count + 1;
        end if;

    end if;
end process;

```

Fonte: elaborado pelos autores (2026).

4.2 Módulo top_nrz_i_basys3.vhd

O módulo de topo do projeto NRZ-I adota uma abordagem arquitetural diferente para o cálculo dos LEDs: em vez de utilizar um processo sequencial para calcular os níveis modulados, emprega uma cadeia combinacional de atribuições condicionais para calcular instantaneamente os 16 níveis do vetor *nivel* a partir dos *switches*. O nível inicial (*nivel(15)*) é simplesmente o valor do *switch* mais significativo; cada nível subsequente é calculado invertendo ou preservando o anterior de acordo com o valor do *switch* correspondente. Os LEDs exibem o vetor *nivel* completo, revelando visualmente o resultado da modulação NRZ-I para todos os 16 bits simultaneamente. O Código 7, a seguir, apresenta essa cadeia combinacional.

**Código 7 – Cadeia combinacional para cálculo dos 16 níveis NRZ-I nos LEDs
(top_nrz_i_basys3.vhd)**

```

-- Nível inicial: primeiro switch define o ponto de partida
nivel(15) <= sw(15);
-- Cada nível seguinte inverte ou preserva o anterior conforme o switch

```

```

nivel(14) <= not nivel(15) when sw(14) = '1' else nivel(15);
nivel(13) <= not nivel(14) when sw(13) = '1' else nivel(14);
nivel(12) <= not nivel(13) when sw(12) = '1' else nivel(13);
-- ... (idem para nivel(11) até nivel(1))
nivel(0)  <= not nivel(1)  when sw(0)  = '1' else nivel(1);

-- LEDs mostram o resultado modulado de todos os 16 bits
led <= nivel;

```

Fonte: elaborado pelos autores (2026).

Para a saída VGA, o módulo seleciona sequencialmente os níveis calculados: a cada 2 segundos, o registrador *nrz_out_reg* é atualizado com *nivel(bit_index)*, exibindo na tela a cor correspondente ao nível modulado do bit atual. A lógica de sincronismo VGA e de cores é idêntica à do módulo NRZ-L: *nrz_out_reg* = '1' → azul; *nrz_out_reg* = '0' → vermelho. Assim, enquanto os LEDs mostram todos os 16 níveis NRZ-I simultaneamente, o monitor exibe um bit de cada vez em ciclos de 2 segundos, percorrendo a sequência de *nivel(15)* até *nivel(0)* continuamente.

5. PADRÃO VGA 640×480 E GERAÇÃO DE SINCRONISMO

A interface VGA (*Video Graphics Array*) é um padrão analógico de transmissão de vídeo que utiliza sinais de sincronismo horizontal (*Hsync*) e vertical (*Vsync*) para sincronizar a varredura do monitor com os dados de cor transmitidos. No modo 640×480 a 60 Hz utilizado neste projeto, a frequência de pixel é de aproximadamente 25,175 MHz, comumente aproximada para 25 MHz. Como a Basys 3 fornece um *clock* de 100 MHz, os módulos de topo implementam um divisor de frequência por 4 - realizado com o contador *pixel_div* de 2 bits - para obter o *clock* de pixel necessário. Os contadores *h_count* e *v_count* são incrementados a cada período de pixel e percorrem, respectivamente, a linha horizontal completa (800 posições) e o quadro vertical completo (525 linhas), incluindo as regiões de *front porch*, pulso de sincronismo e *back porch*. A Tabela 2, a seguir, resume os parâmetros de temporização utilizados.

Tabela 2 – Parâmetros de temporização do padrão VGA 640×480 @ 60 Hz

PARÂMETRO	HORIZONTAL (<i>h_count</i>)	VERTICAL (<i>v_count</i>)
Área visível	0 – 639 (640 pixels)	0 – 479 (480 linhas)
Front porch	640 – 655 (16 pixels)	480 – 489 (10 linhas)
Pulso de Sync (ativo-baixo)	656 – 751 (96 pixels)	490 – 491 (2 linhas)

Back porch	752 – 799 (48 pixels)	492 – 524 (33 linhas)
Total por quadro	800 posições	525 linhas
Frequência resultante	$\sim 25 \text{ MHz} / 800 = \sim 31,25 \text{ kHz}$	$\sim 31,25 \text{ kHz} / 525 \approx 59,5 \text{ Hz}$

Fonte: elaborado pelos autores (2026).

O sinal *video_on* é '1' apenas quando ambos os contadores estão dentro da área visível (*h_count* < 640 and *v_count* < 480). Fora dessa região, os sinais de cor são forçados a zero ('0000'), o que garante que nenhuma cor seja transmitida durante os pulsos de sincronismo ou nas regiões de *porch*. O Código 8, a seguir, apresenta a geração dos sinais de sincronismo, da área visível e das cores.

Código 8 – Geração de sincronismo VGA, área visível e lógica de cores (top_nrz_l_basys3.vhd)

```
-- Divisor de clock: 100 MHz / 4 = 25 MHz (pixel clock)
pixel_div <= pixel_div + 1;
if pixel_div = "11" then -- a cada 4 ciclos, atualiza contadores
    if h_count = 799 then h_count <= 0;
        if v_count = 524 then v_count <= 0;
            else v_count <= v_count + 1; end if;
        else h_count <= h_count + 1; end if;
    end if;

-- Área visível
video_on <= '1' when (h_count < 640 and v_count < 480) else '0';

-- Sincronismo (ativo-baixo)
Hsync <= '0' when (h_count >= 656 and h_count < 752) else '1';
Vsync <= '0' when (v_count >= 490 and v_count < 492) else '1';

-- Cores: nrz_out_reg='1' -> azul; nrz_out_reg='0' -> vermelho
vgaRed   <= "1111" when (video_on='1' and nrz_out_reg='0') else "0000";
vgaGreen <= "0000";
vgaBlue  <= "1111" when (video_on='1' and nrz_out_reg='1') else "0000";
```

Fonte: elaborado pelos autores (2026).

A ausência de verde (*vgaGreen* <= "0000") é intencional: as cores vermelho puro e azul puro foram escolhidas para maximizar o contraste visual e tornar a distinção entre os níveis da modulação imediatamente perceptível a qualquer observador. O vetor de 4 bits para cada canal de cor assume o valor "1111" (máxima intensidade) ou "0000" (apagado), resultando em telas uniformemente coloridas sem gradientes.

6. ARQUIVO DE RESTRIÇÕES XDC

O arquivo de restrições XDC deste projeto é mais extenso do que o das versões anteriores, pois além dos pinos de *switches* e LEDs, inclui o mapeamento do *clock* de 100 MHz e de todos os pinos VGA da placa Basys 3. O mapeamento do *clock* é especialmente importante: além da instrução *set_property PACKAGE_PIN W5*, que associa o sinal *clk* ao pino W5 (oscilador de 100 MHz da placa), é necessário declarar a restrição de temporização com *create_clock*, especificando o período de 10 ns e a forma de onda com 50% de ciclo de trabalho. Sem essa declaração, o Vivado não consegue executar a análise de temporização e pode emitir avisos ou erros durante a implementação. A Tabela 3, a seguir, apresenta o mapeamento dos pinos VGA, e o Código 9 mostra o trecho do arquivo XDC com o clock e os pinos VGA.

Tabela 3 – Mapeamento dos pinos VGA no arquivo de restrições XDC

SINAL VHDL	PINO (BASYS 3)	CANAL VGA
vgaRed[0..3]	G19, H19, J19, N19	Vermelho (4 bits)
vgaGreen[0..3]	J17, H17, G17, D17	Verde (4 bits)
vgaBlue[0..3]	N18, L18, K18, J18	Azul (4 bits)
Hsync	P19	Sincronismo horizontal
Vsync	R19	Sincronismo vertical
clk	W5	Clock 100 MHz (oscilador da placa)

Fonte: elaborado pelos autores (2026).

Código 9 – Trecho do arquivo de restrições basys3_nrz_l.xdc: clock e pinos VGA

```
## Clock de 100 MHz da Basys 3
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports clk]

## Vermelho
set_property PACKAGE_PIN G19 [get_ports {vgaRed[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[0]}]
set_property PACKAGE_PIN N19 [get_ports {vgaRed[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vgaRed[3]}]

## Azul
set_property PACKAGE_PIN N18 [get_ports {vgaBlue[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vgaBlue[0]}]

## Sincronismo
set_property PACKAGE_PIN P19 [get_ports Hsync]
set_property IOSTANDARD LVCMOS33 [get_ports Hsync]
set_property PACKAGE_PIN R19 [get_ports Vsync]
set_property IOSTANDARD LVCMOS33 [get_ports Vsync]
```

Fonte: elaborado pelos autores (2026).

7. CRIAÇÃO DO PROJETO NO VIVADO

Abra o Vivado 2025.x e, na tela inicial, selecione *Create Project*. O assistente de criação solicitará, em etapas sucessivas, o nome do projeto, o diretório de armazenamento e o tipo de projeto. Crie projetos separados para cada modulação, denominados respectivamente *NRZ_L_VGA* e *NRZ_I_VGA*, armazenados em diretórios distintos para evitar conflito entre os arquivos gerados pelo fluxo de síntese e implementação. Na etapa de tipo de projeto, selecione *RTL Project* e avance até a seleção do dispositivo. Na aba *Boards*, pesquise e selecione a placa *Basys3* da Digilent; caso não apareça, selecione diretamente o dispositivo *xc7a35tcbg236-1* (família Artix-7, encapsulamento CPG236, grau de velocidade -1). Clique em *Finish* para concluir a criação.

8. ADIÇÃO DOS ARQUIVOS VHDL E RESTRIÇÕES

Com o projeto criado e aberto, acesse *File* → *Add Sources* e selecione *Add or Create Design Sources* para adicionar os arquivos VHDL de hardware. Para o projeto NRZ-L, adicione *nrz_l.vhd* e *top_nrz_l_basys3.vhd*, certificando-se de que o módulo de topo aparece com o ícone de raiz hierárquica no painel *Sources*. Para o projeto NRZ-I, adicione *nrz_i.vhd* e *top_nrz_i_basys3.vhd*. Em seguida, repita o processo com a opção *Add or Create Simulation Sources* para incluir os *testbenches* (*tb_nrz_l.vhd* ou *tb_nrz_i.vhd*), que devem aparecer na seção *Simulation Sources*. Por fim, adicione o arquivo de restrições XDC correspondente via *Add or Create Constraints*; ele aparecerá na seção *Constraints* e é obrigatório para a implementação em hardware, pois mapeia o *clock*, os *switches*, os LEDs e todos os pinos VGA.

9. CONFIGURAÇÃO DA SIMULAÇÃO

A simulação comportamental é executada sobre os módulos de baixo nível (*nrz_l.vhd* e *nrz_i.vhd*) e não sobre os módulos de topo com VGA. Isso ocorre porque o módulo de topo utiliza $CYCLES_PER_BIT = 200.000.000$, o que tornaria a simulação inviável - seriam necessários $3,2 \times 10^{10}$ nanosegundos (32 segundos simulados) para observar a transmissão completa. Os módulos de simulação utilizam $CYCLES_PER_BIT = 5$, resultando em uma transmissão completa de apenas 1.600 ns, perfeitamente visualizável no simulador. Para configurar a simulação, verifique no painel *Sources* que o *testbench* correspondente (*tb_nrz_l* ou *tb_nrz_i*) está selecionado como raiz da simulação em *Simulation Sources*. Caso necessário, clique com o botão direito sobre o *testbench* e selecione *Set as Top*.

Para ajustar o tempo de simulação, acesse *Simulation Settings* (clique com o botão direito em *Simulation* no painel *Flow Navigator* e escolha *Simulation Settings*). Defina o tempo de execução como *2000 ns* - valor suficiente para cobrir a transmissão completa de 1.600 ns e os 100 ns adicionais antes do encerramento com *assert failure*. Esse valor corresponde ao campo *xsim.simulate.runtime* nas configurações de simulação do Vivado.

10. EXECUÇÃO DA SIMULAÇÃO

Com a configuração concluída, clique em *Run Simulation* → *Run Behavioral Simulation* no painel *Flow Navigator*. O Vivado compilará os arquivos VHDL, elaborará o *snapshot* do *testbench* e abrirá a janela de *waveform*. Na janela de *waveform*, adicione os sinais *clk*, *rst*, *start*, *data_in*, *nrz_out* e *bit_indx* - todos disponíveis no painel *Scope* sob a instância *uut*. Utilize o botão *Run For* com o valor *2000 ns* para executar toda a simulação de uma vez. O *assert failure* ao final encerra a simulação automaticamente com a mensagem "*Fim da simulacao NRZ-L*" (ou NRZ-I), confirmando que todos os bits foram processados corretamente.

11. RESULTADOS EM HARDWARE — SAÍDA VGA

Após a geração do *bitstream* e a programação da placa Basys 3 via *Hardware Manager*, o monitor conectado à porta VGA da placa exibe a sequência de cores que representa a forma de onda modulada. Para o projeto NRZ-L, cada bit da sequência configurada pelos *switches* é exibido na tela por 2 segundos: *switches* ligados (bit '1') produzem tela **azul**; *switches* desligados (bit '0') produzem tela **vermelha**. A sequência percorre *sw[15]*, *sw[14]*, ..., *sw[0]* e reinicia continuamente. A Tabela 4, a seguir, resume a correspondência entre nível modulado e cor exibida.

Tabela 4 – Correspondência entre o nível modulado e a cor exibida na saída VGA

NÍVEL MODULADO (nrz_out_reg)	vgaRed	vgaGreen	vgaBlue	COR DA TELA
'1' (nível alto)	"0000" (apagado)	"0000" (apagado)	"1111" (máx.)	Azul puro
'0' (nível baixo)	"1111" (máx.)	"0000" (apagado)	"0000" (apagado)	Vermelho puro

Fonte: elaborado pelos autores (2026).

12. IDENTIFICAÇÃO DOS SINAIS

Em ambos os projetos, a entrada de bits é definida pelos 16 *switches* da placa Basys 3 (*sw[15:0]*), cujo estado é lido ciclicamente a cada intervalo de bit. O sinal de saída modulada é representado, em hardware, pelo registrador *nrz_out_reg*, que comanda diretamente a cor exibida no monitor. Adicionalmente, os LEDs da placa fornecem uma segunda representação visual: no projeto NRZ-L, exibem o estado atual dos *switches* (entrada de bits); no projeto NRZ-I, exibem a sequência modulada completa calculada pela cadeia combinacional - ou seja, os 16 níveis resultantes da aplicação da regra de inversão sobre todos os *switches* simultaneamente. A Tabela 5, a seguir, sintetiza essa identificação.

Tabela 5 – Identificação dos sinais nos projetos NRZ-L e NRZ-I com VGA

SINAL	PROJETO NRZ-L	PROJETO NRZ-I
Entrada de bits (<i>sw[15:0]</i>)	Switch ON = bit '1'; switch OFF = bit '0'. LEDs mostram os switches.	Switch ON = bit '1'; switch OFF = bit '0'. LEDs mostram o resultado NRZ-I de todos os bits.
Saída modulada (<i>nrz_out_reg</i>)	Nível alto → tela azul. Nível baixo → tela vermelha.	Nível alto → tela azul. Nível baixo → tela vermelha.
<i>bit_index</i> (contador)	Índice do switch sendo exibido no momento (15 → 0, ciclicamente).	Índice do nível NRZ-I sendo exibido no momento (15 → 0, ciclicamente).
LEDs (<i>led[15:0]</i>)	Cópia direta dos switches: <i>led</i> ≤ <i>sw</i> .	Resultado combinacional da modulação NRZ-I: <i>led</i> ≤ nível.

Fonte: elaborado pelos autores (2026).

13. EXPLICAÇÃO DO COMPORTAMENTO OBSERVADO

No projeto NRZ-L, o comportamento da tela VGA é uma representação visual direta da sequência de bits definida pelos *switches*. A cada 2 segundos a cor se altera conforme o próximo bit: se *sw[14]* estiver ligado (bit '1'), a tela exibida durante o intervalo desse bit será azul; se desligado (bit '0'), vermelha. O tempo total em cada cor é proporcional à quantidade de '1's e '0's configurados - com todos os *switches* ligados, a tela permanece azul ininterruptamente; com todos desligados, permanece vermelha. A transição entre as cores é abrupta e síncrona com o *clock*, ocorrendo exatamente no instante em que *cycle_count* atinge zero e *bit_index* avança para o próximo *switch*.

No projeto NRZ-I, o comportamento é conceitualmente diferente e visualmente mais dinâmico para sequências com muitos bits '1'. Como cada bit '1' provoca uma inversão do nível

anterior, sequências longas de '1's consecutivos geram alternâncias azul→vermelho→azul→vermelho a cada 2 segundos, enquanto bits '0' interrompem essa alternância mantendo a cor atual. Os LEDs do NRZ-I mostram, de forma estática, o resultado completo da modulação: a cadeia combinacional $nivel(14) \leq not\ nivel(15)\ when\ sw(14)='1'$ $else\ nivel(15)$ - e assim por diante até $nivel(0)$ - calcula e exibe instantaneamente, em todos os 16 LEDs ao mesmo tempo, quais posições da sequência resultam em nível alto e quais resultam em nível baixo, permitindo uma visão panorâmica do sinal NRZ-I antes mesmo de aguardar 32 segundos pelo ciclo completo da tela.

14. COMPARAÇÃO ENTRE NRZ-L E NRZ-I

Ambas as modulações pertencem à família NRZ (*Non-Return-to-Zero*), na qual o sinal permanece em um nível constante durante todo o intervalo do símbolo. A distinção fundamental - que se manifesta tanto na forma de onda simulada quanto no comportamento da tela VGA - é o que cada bit representa: no NRZ-L, o bit codifica o nível absoluto do sinal, enquanto no NRZ-I, o bit codifica uma ação sobre o nível anterior. Essa diferença tem consequências diretas e observáveis: com a mesma sequência de entrada, os dois projetos podem produzir cores completamente diferentes na tela ao longo do ciclo, pois o NRZ-I depende do histórico de inversões acumuladas desde o primeiro bit.

Tabela 6 – Comparação entre as modulações NRZ-L e NRZ-I na implementação com VGA

CRITÉRIO	NRZ-L (com VGA)	NRZ-I (com VGA)
Representação do bit '1'	Tela azul durante 2 s.	Troca de cor a cada 2 s (se anterior era azul, passa para vermelho, e vice-versa).
Representação do bit '0'	Tela vermelha durante 2 s.	Mantém a cor atual durante 2 s (sem transição de cor).
Sequência todo-1's	Tela azul contínua.	Alternância azul↔vermelho a cada 2 s.
Sequência todo-0's	Tela vermelha contínua.	Tela estática na cor do primeiro bit (sem transições).
Sequência 10101...	Tela alternando azul↔vermelho a cada 2 s.	Depende do estado inicial; comportamento idêntico ao todo-1's se inicializa em '1'.
Leitura direta do bit pela cor	Sim: azul = '1', vermelho = '0'.	Não: a cor não representa o bit diretamente, mas o nível acumulado de inversões.

LEDs	Mostram os switches (entrada).	Mostram o resultado modulado de todos os bits simultaneamente (saída completa).
Implementação VHDL	led <= sw; sequencial com CYCLES_PER_BIT.	Cadeia combinacional nivel(N) + sequencial com CYCLES_PER_BIT.

Fonte: elaborado pelos autores (2026).

15. INTERPRETAÇÃO FINAL DOS RESULTADOS

A implementação com saída VGA acrescenta uma dimensão pedagógica inédita nas versões anteriores. Ao observar a alternância de cores, o usuário percebe diretamente a duração de cada bit - 2 segundos de azul ou vermelho -, tornando a relação entre sequência de entrada e sinal modulado imediata e imediata. O NRZ-L demonstra claramente sua natureza **estacionária** - a cor da tela em qualquer instante é simplesmente o valor do *switch* correspondente -, enquanto o NRZ-I demonstra sua natureza **diferencial**: é impossível determinar o bit atual apenas pela cor da tela sem conhecer a cor anterior, o que ilustra na prática o conceito de codificação por transição.

Do ponto de vista técnico, este projeto introduz dois elementos novos em relação às versões anteriores: a geração de sincronismo VGA e a declaração de restrição de temporização (*create_clock*) no arquivo XDC. A geração de sincronismo demonstra como um padrão analógico amplamente utilizado - o VGA - pode ser implementado inteiramente em VHDL com poucos contadores e atribuições condicionais. A restrição de temporização habilita a análise de temporização estática no Vivado, garantindo que o caminho crítico do circuito respeite o período de 10 ns do *clock* da Basys 3. A combinação desses elementos em um único projeto - modulação digital, divisor de *clock*, sincronismo VGA e lógica de cores - demonstra a capacidade da FPGA de integrar múltiplos domínios de funcionalidade em um único dispositivo programável.

REFERÊNCIAS BIBLIOGRÁFICAS

- SINGH, Gurashish; GUPTA, Ankit; GUPTA, Kirti; PANDEY, Neeta. FPGA implementation of different NRZ line coding schemes. In: INDIA INTERNATIONAL CONFERENCE ON INFORMATION PROCESSING, 1., 2016, Nova Delhi. Proceedings [...]. Piscataway: IEEE, 2016. p. 1-7. DOI: 10.1109/IICIP.2016.7975311. Disponível em: <https://ieeexplore.ieee.org/document/7975311>. Acesso em: 23 maio 2026.
- DIGILENT, Inc. Basys 3 Artix-7 FPGA Trainer Board: Reference Manual. Digilent, 2023. Disponível em: <https://digilent.com/reference/programmable-logic/basys-3/reference-manual>. Acesso em: 23 maio 2026.